

Agent 部署与发布：从灰度到回滚

语言策略：code。正文三语言一致；服务骨架与容器化示例按 Java 生态实现。地图节点：04 开发与工程化 / 部署与发布。配套文档：[Agent 可观测与追踪](#)、[Agent 评测体系](#)。

0. 结论先说

- Agent 的发布单元不止代码：**Prompt、工具、模型路由、记忆策略都会改变线上行为**，必须一起版本化。
- 推荐路径：离线回归 → 影子流量 → 灰度 → 全量；每个阶段都有可观测指标和回滚条件。
- 部署前先回答三个问题：任务能不能幂等？跑一半断了怎么办？回滚时旧 Prompt 和旧工具是否还在？

1. 部署形态

形态	适用场景	注意点
同步 API 服务	交互式助手	需要流式、超时与取消
异步任务队列	长任务、批处理	任务状态、幂等、死信队列
定时/事件驱动 Worker	巡检、告警处理	并发控制、去重
边端/混合部署	数据不出域	模型版本与配置下发

2. 发布单元与版本化

发布单元	版本标识	回滚方式
应用代码	Git commit / 镜像 tag	回滚镜像
System Prompt	prompt_version	配置回滚
工具 Schema / Skill	toolset_version	注册表回滚
模型路由策略	route_version	网关配置回滚
评测集与门禁	eval_version	随发布记录关联

发布包必须记录以上五个版本，Trace 中也要带上 prompt_version 与 toolset_version，否则线上问题无法归因。

3. 发布流水线

1 代码/配置变更
2
3
4 离线回归（成功率和工具准确率门禁）
5
6
7 影子流量（只记录，不影响用户）
8
9
10 灰度 5% → 25% → 100%
11
12
13 观察 P95、成功率、成本、危险动作率
14
15
16 通过则全量，否则自动回滚

灰度期间高危工具默认只读或审批升级；灰度时间不少于一个完整业务周期。

4. 运行时要求

- 健康检查：/health 同时检查模型供应商、向量库和工具依赖；
- 超时与取消：Agent Loop 总超时、单工具超时、用户取消；

- 幂等：同一 task_id 重复提交只执行一次；
- 限流：按用户、租户、工具设置配额；
- 审计：部署事件、配置变更、回滚事件全部留痕。

5. Java 版服务骨架与容器化

```
1 public record AgentRunRequest(String taskId, String input, String
2   ↪ promptVersion, String userId) {}
3 public record AgentRunResponse(String taskId, String status, String
4   ↪ result, long elapsedMs) {}
5 public record HealthStatus(String status, boolean modelUp, boolean
6   ↪ vectorDbUp, boolean toolsUp) {}
7
8 public interface AgentRuntime {
9     AgentRunResponse run(AgentRunRequest request);
10    HealthStatus health();
11 }
12
13 public class AgentService implements AgentRuntime {
14     private final AgentRuntime delegate;
15
16     public AgentService(AgentRuntime delegate) { this.delegate =
17       ↪ delegate; }
18
19     @Override
20     public AgentRunResponse run(AgentRunRequest request) {
21         // 生产实现：先做幂等校验，再进入 Agent Loop，超时或异常返回可
22       ↪ 重试状态
23         return delegate.run(request);
24     }
25
26     @Override
27     public HealthStatus health() {
28         return delegate.health();
29     }
30 }
```

```
1 FROM eclipse-temurin:21-jre
2 WORKDIR /app
3 COPY target/agent-service.jar app.jar
4 EXPOSE 8080
5 ENTRYPOINT ["java", "-Xmx1g", "-jar", "app.jar"]
```

```
1 健康检查: GET /health
2 发布命令: docker build -t agent-service:<git-sha> . && docker push
3   ↪ agent-service:<git-sha>
4 回滚命令: kubectl set image deploy/agent-service agent-service=agent
5   ↪ -service:<previous-sha>
```

6. 回滚决策表

信号	阈值示例	动作
任务成功率	低于基线 5 个百分点	立即回滚
P95 延迟	超过 SLO 的 120	
危险动作率	出现一次未授权高危动作	冻结 + 回滚
单任务成本	超过预算 130	
用户投诉	同主题短时激增	暂停灰度

7. 上线 Runbook 检查单

发布包记录了代码、Prompt、工具、模型路由、评测集五个版本；

离线回归门禁通过；

回滚镜像和旧配置可用；

高危工具已配置审批；

监控面板和告警已就绪；

灰度比例、时长、回滚触发条件已明确；

事故联系人已确认。

8. 延伸阅读

- [大模型网关](#)

- [Agent 可观测与追踪](#)
- [AI 应用系统设计](#)